SOFTWARE ENGINEERING COURSE

LECTURE 4

Requirements Engineering

MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 1	September 2 — September 6, 2024	Course Introduction	None
Week 2	September 9 — September 13, 2024	Introduction to Software Engineering	Ch. 1
Week 3	September 16 — September 20, 2024	Software Development Life Cycle & Methodologies	Ch. 2
Week 4	September 23 — September 28, 2024	Software Development Processes	Ch. 3
Week 5	September 30 — October 4, 2024	Requirements Engineering	Ch. 4
Week 6	October 7 — October 11, 2024	Software Modelling, Coding and Dev.	Ch. 5
Week 7	October 14 — October 18, 2024	Software Architectural Design	Ch. 6

LEARNING OBJECTIVES

At the end of this lecture, you will be able to demonstrate an understanding and explain:

- Functional and non-functional requirements
- Requirements engineering processes
- Requirements elicitation
- Requirements specification
- Requirements validation
- Requirements change

REQUIREMENTS ENGINEERING

- The process of establishing the services that a customer requires from a system and the constraints under which the system operates and developed.
- The system requirements are the descriptions of the system services and constraints that are generated during the requirements engineering process.

WHAT IS REQUIREMENTS ENGINEERING

- The process of formalizing and defining needs at each stage of system design, including customer requirements, design requirements, and testing requirements.
- A systematic and strict approach to the definition, creation, and verification of requirements for a software system.
- It ensures that the software meets the needs of its users, is delivered on time, within budget, and to the required quality standards.

WHAT IS A REQUIREMENT?

- It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.
- This is inevitable as requirements may serve a dual function
  - May be the basis for a bid for a contract — therefore must be open to interpretation;
  - May be the basis for the contract itself - therefore must be defined in detail;
  - Both these statements may be called requirements.

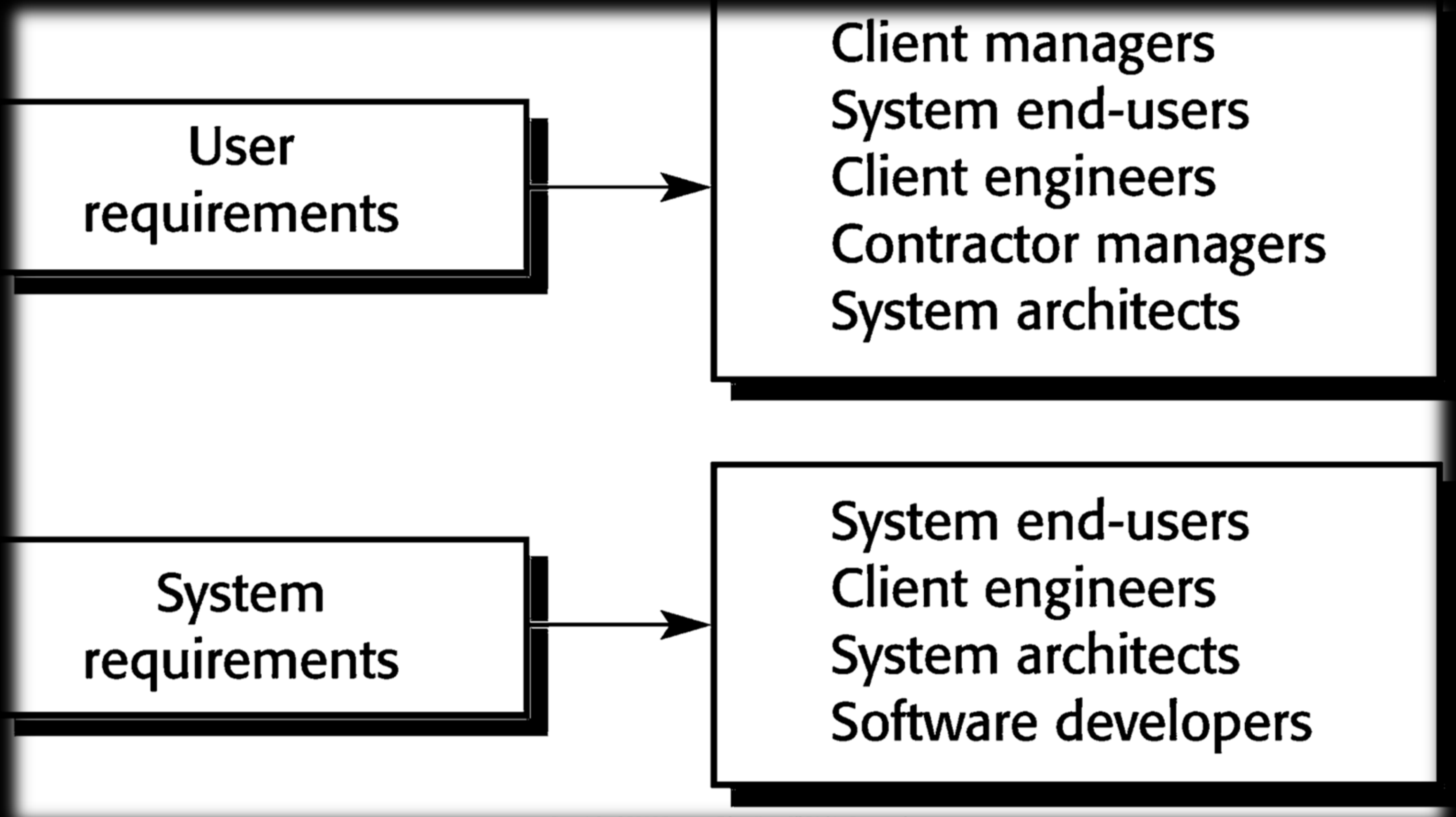
REQUIREMENTS ABSTRACTION (DAVIS)

“If a company wishes to let a contract for a large software development project, it must define its needs in a sufficiently abstract way that a solution is not pre-defined. The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization’s needs. Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do. Both of these documents may be called the requirements document for the system.”

TYPES OF REQUIREMENT

- **User requirements**
  - Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- **System requirements**
  - A structured document setting out detailed descriptions of the system’s functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

READERS OF DIFFERENT TYPES OF REQUIREMENTS



SYSTEM STAKEHOLDERS

- Any person or organization who is affected by the system in some way and so who has a legitimate interest
- Stakeholder types
  - End users
  - System managers
  - System owners
  - External stakeholders

AGILE METHODS AND REQUIREMENTS

- Many agile methods argue that producing detailed system requirements is a waste of time as requirements change so quickly.
- The requirements document is therefore always out of date.
- Agile methods usually use incremental requirements engineering and may express requirements as ‘user stories’.
- This is practical for business systems but problematic for systems that require pre-delivery analysis or systems developed by several teams.

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional requirements

- Statements of services the system should provide, how the system should react to particular inputs and how the system should behave
- May state what the system should not do.

Non-functional requirements

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards
- Often apply to the system rather than individual features or services.

Domain requirements

- Constraints on the system from the domain of operation

FUNCTIONAL REQUIREMENTS

- Describe functionality or system services.
- Depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

REQUIREMENTS COMPLETENESS AND CONSISTENCY

- In principle, requirements should be both complete and consistent.
- **Complete**
  - They should include descriptions of all facilities required.
- **Consistent**
  - There should be no conflicts or contradictions in the descriptions of the system facilities.
- In practice, because of system and environmental complexity, it is impossible to produce a complete and consistent requirements document.

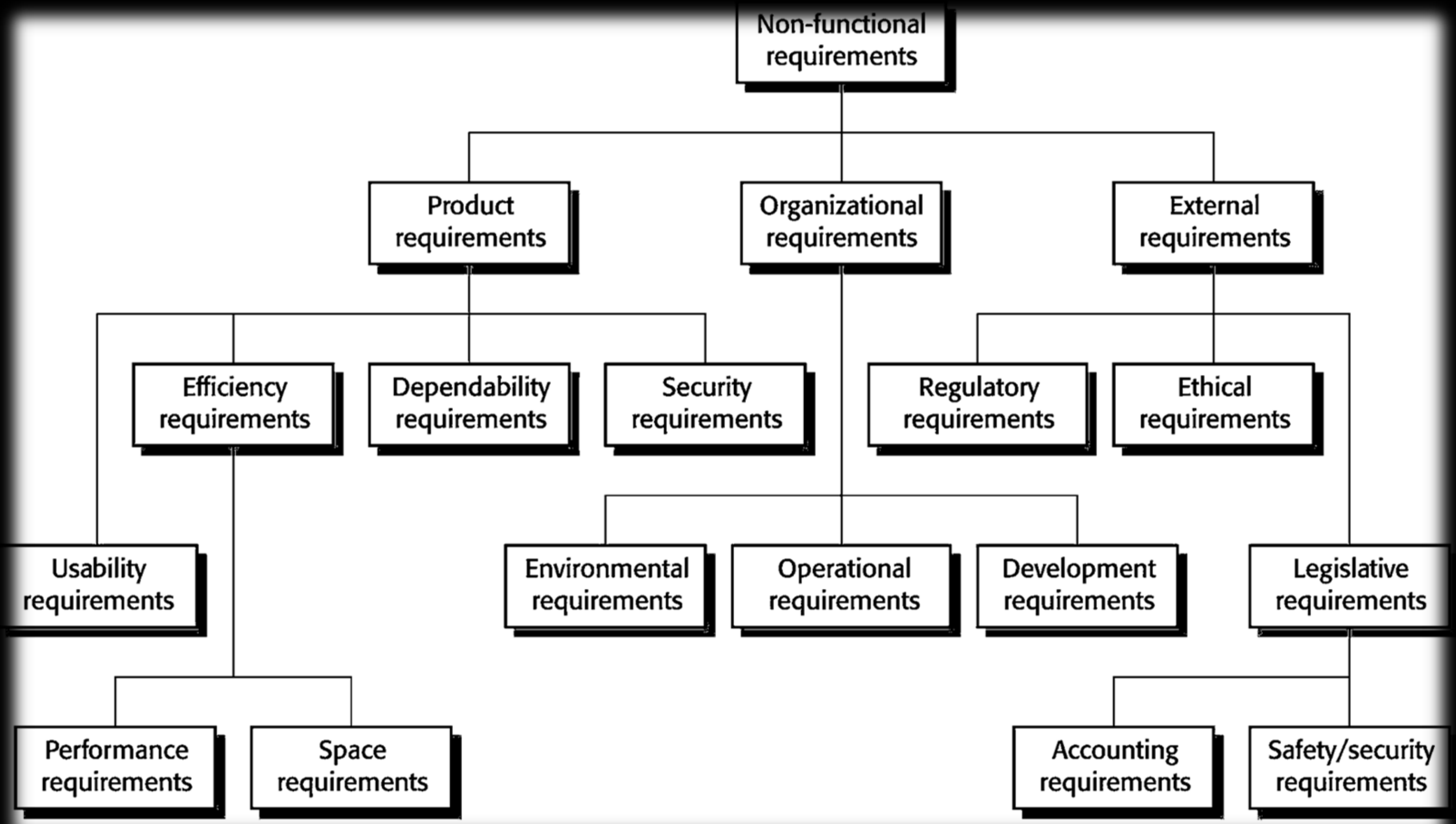
NON-FUNCTIONAL REQUIREMENTS

- These define system properties and constraints response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified mandating a particular IDE, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.

NON-FUNCTIONAL REQUIREMENTS IMPLEMENTATION

- Non-functional requirements may affect the overall architecture of a system rather than the individual components.
  - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
  - It may also generate requirements that restrict existing requirements.

TYPES OF NONFUNCTIONAL REQUIREMENT



NON-FUNCTIONAL CLASSIFICATIONS

- Product requirements
  - Requirements which specify that the delivered product must behave in a particular way
- Organisational requirements
  - Requirements which are a consequence of organisational policies and procedures
- External requirements
  - Requirements which arise from factors which are external to the system and its development process

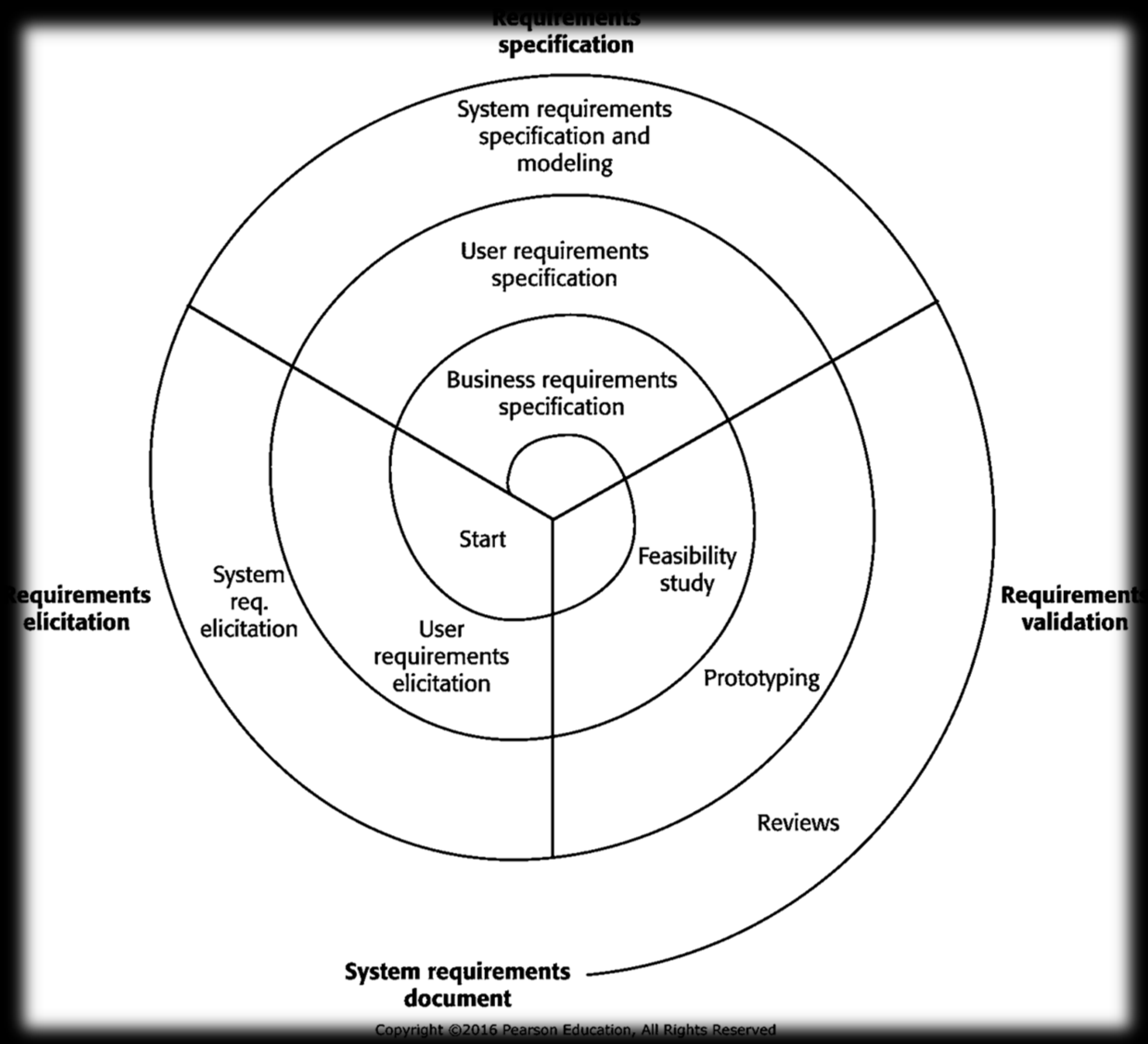
GOALS AND REQUIREMENTS

- Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify.
- Goal
  - A general intention of the user such as ease of use.
- Verifiable non-functional requirement
  - A statement using some measure that can be objectively tested.
- Goals are helpful to developers as they convey the intentions of the system users.

REQUIREMENTS ENGINEERING PROCESSES

- The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements.
- However, there are a number of generic activities common to all processes
  - Requirements elicitation;
  - Requirements analysis;
  - Requirements validation;
  - Requirements management.
- In practice, RE is an iterative activity in which these processes are interleaved.

A SPIRAL VIEW OF THE RE PROCESS



REQUIREMENTS ELICITATION AND ANALYSIS

- Sometimes called requirements elicitation or requirements discovery.
- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc.

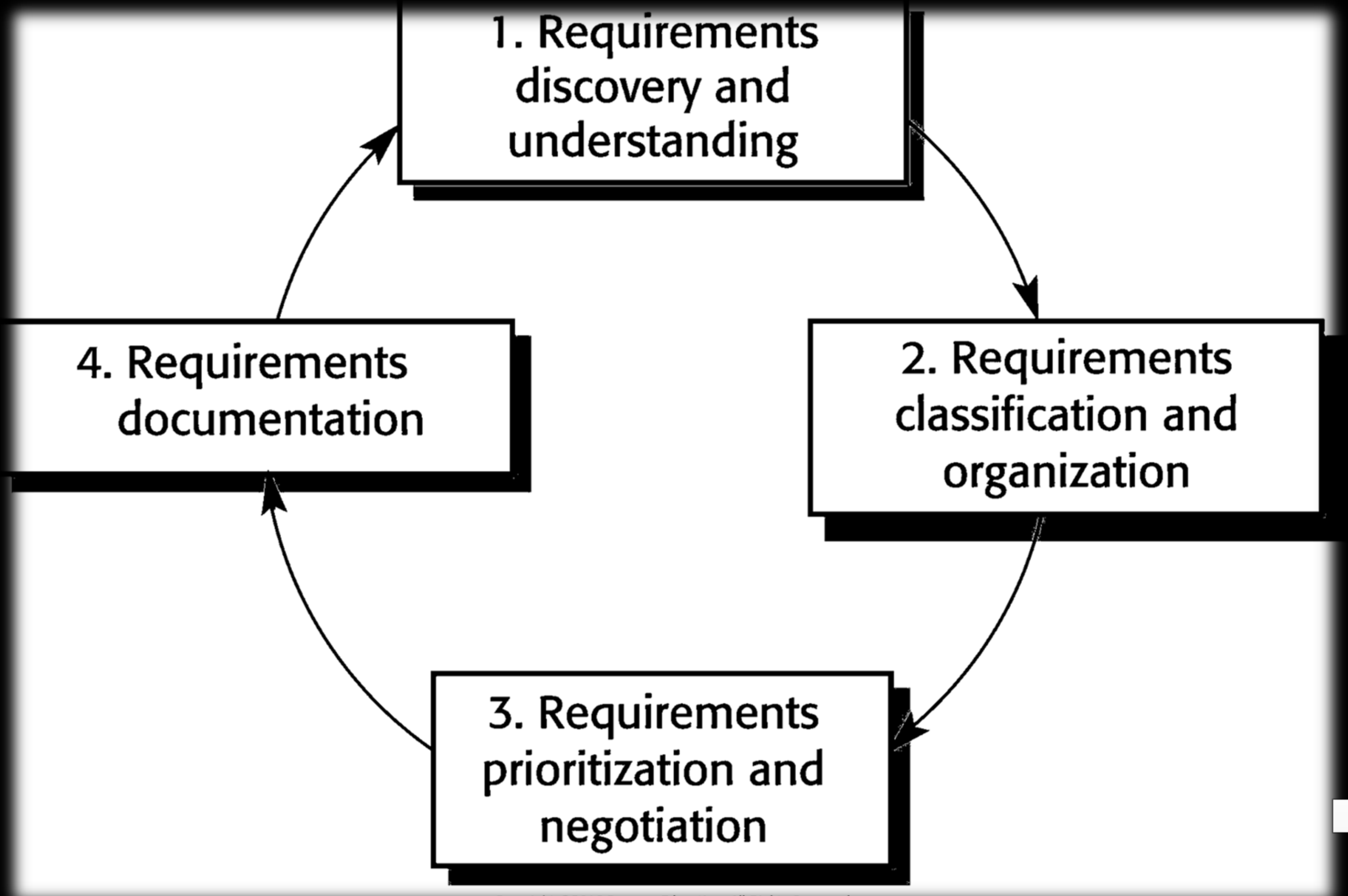
PROBLEMS OF REQUIREMENTS ELICITATION

- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

PROCESS ACTIVITIES

- Requirements discovery
  - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- Requirements classification and organisation
  - Groups related requirements and organises them into coherent clusters.
- Prioritisation and negotiation
  - Prioritising requirements and resolving requirements conflicts.
- Requirements specification
  - Requirements are documented and input into the next round of the spiral.

REQUIREMENTS ELICITATION AND ANALYSIS PROCESS



REQUIREMENTS DISCOVERY

- The process of gathering information about the required and existing systems and distilling the user and system requirements from this information.
- Interaction is with system stakeholders from managers to external regulators.
- Systems normally have a range of stakeholders.

INTERVIEWING

- Formal or informal interviews with stakeholders are part of most RE processes.
- Types of interview
  - Closed interviews based on pre-determined list of questions
  - Open interviews where various issues are explored with stakeholders.

INTERVIEWS IN PRACTICE

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviewers need to be open-minded without pre-conceived ideas of what the system should do
- You need to prompt the use to talk about the system by suggesting requirements rather than simply asking them what they want.

PROBLEMS WITH INTERVIEWS

- Application specialists may use language to describe their work that isn't easy for the requirements engineer to understand.
- Interviews are not good for understanding domain requirements
  - Requirements engineers cannot understand specific domain terminology;
  - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

ETHNOGRAPHY

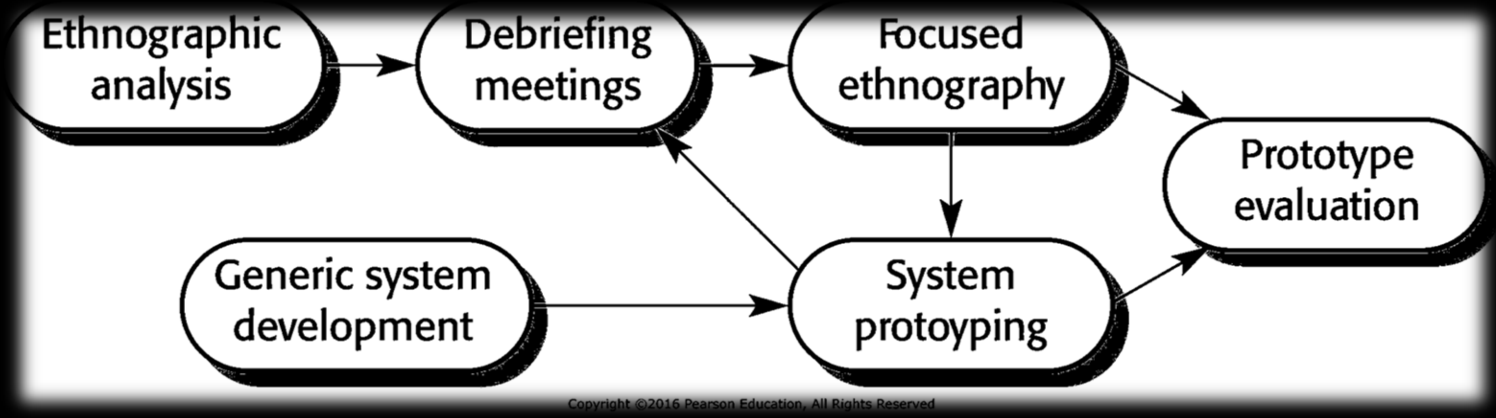
- A social scientist spends a considerable time observing and analysing how people actually work.
- People do not have to explain or articulate their work.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

SCOPE OF ETHNOGRAPHY

- Requirements that are derived from the way that people actually work rather than the way I which process definitions suggest that they ought to work.
- Requirements that are derived from cooperation and awareness of other people’s activities.
  - Awareness of what other people are doing leads to changes in the ways in which we do things.
- Ethnography is effective for understanding existing processes but cannot identify new features that should be added to a system.

FOCUSED ETHNOGRAPHY

- Eethnography could be combined with prototyping



- Prototype development results in unanswered questions which focus the ethnographic analysis.
- The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

STORIES AND SCENARIOS

- Scenarios and user stories are real-life examples of how a system can be used.
- Stories and scenarios are a description of how a system may be used for a particular task.
- Because they are based on a practical situation, stakeholders can relate to them and can comment on their situation with respect to the story.

SCENARIOS

- A structured form of user story
- Scenarios should include
  - A description of the starting situation;
  - A description of the normal flow of events;
  - A description of what can go wrong;
  - Information about other concurrent activities;
  - A description of the state when the scenario finishes.

REQUIREMENTS SPECIFICATION

- The process of writing donw the user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
    - It is therefore important that these are as complete as possible.

WAYS OF WRITING A SYSTEM REQUIREMENTS SPECIFICATION

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document

REQUIREMENTS AND DESIGN

- In principle, requirements should state what the system should do and the design should describe how it does this.
- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
  - This may be the consequence of a regulatory requirement.

NATURAL LANGUAGE SPECIFICATION

- Requirements are written as natural language sentences supplemented by diagrams and tables.
- Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

GUIDELINES FOR WRITING REQUIREMENTS

- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

STRUCTURED SPECIFICATIONS

- An approach to writing requirements where the freedom of the requirements writer is limited and requirements are written in a standard way.
- This works well for some types of requirements but is sometimes too rigid for writing business system requirements.

FORM-BASED SPECIFICATIONS

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Information about the information needed for the computation and other entities used.
- Description of the action to be taken.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function.

USE CASES

- Use-cases are a kind of scenario that are included in the UML.
- Use cases identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- High-level graphical model supplemented by more detailed tabular description
- UML sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

THE SOFTWARE REQUIREMENTS DOCUMENT

- The software requirements document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

REQUIREMENTS VALIDATION

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

REQUIREMENTS CHECKING

- Validity. Does the system provide the functions which best support the customer’s needs?
- Consistency. Are there any requirements conflicts?
- Completeness. Are all functions required by the customer included?
- Realism. Can the requirements be implemented given available budget and technology
- Verifiability. Can the requirements be checked?

REQUIREMENTS VALIDATION TECHNIQUES

- Requirements reviews
  - Systematic manual analysis of the requirements.
- Prototyping
  - Using an executable model of the system to check requirements.
- Test-case generation
  - Developing tests for requirements to check testability.

CHANGING REQUIREMENTS

- The business and technical environment of the system always changes after installation.
- The people who pay for a system and the users of that system are rarely the same people.
- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.
- The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

REQUIREMENTS MANAGEMENT

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge as a system is being developed and after it has gone into use.
- You need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes. You need to establish a formal process for making change proposals and linking these to system requirements.

REQUIREMENTS CHANGE MANAGEMENT

Change management involves the decision if a requirements change should be accepted

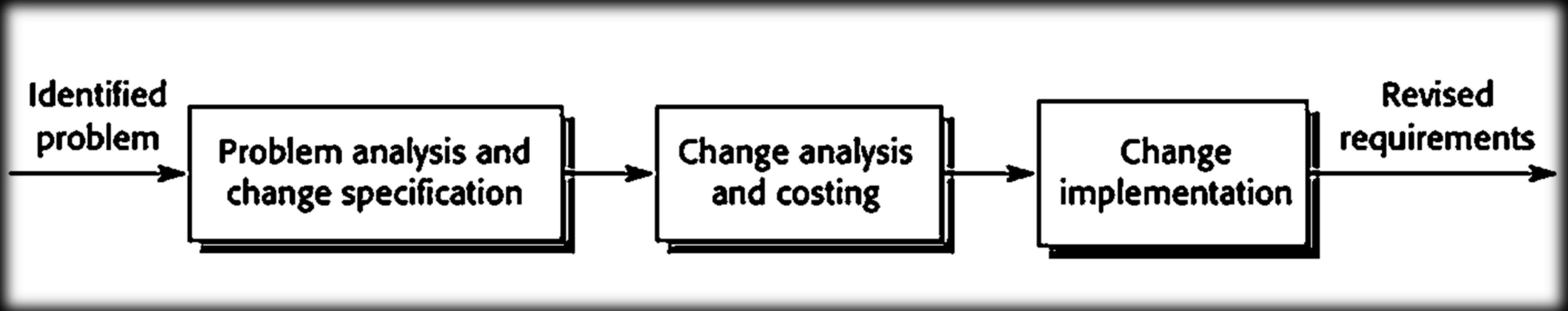
- Problem analysis and change specification
  - During this stage, the problem or the change proposal is analyzed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.

- Change analysis and costing
  - The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.

REQUIREMENTS CHANGE MANAGEMENT

Change implementation

The requirements document and, where necessary, the system design and implementation, are modified. Ideally, the document should be organized so that changes can be easily implemented



CONCLUSION

- As the software project develops and new information becomes available, the iterative requirements engineering process may involve going back and reviewing earlier phases. Throughout the process, stakeholders in the software project must effectively communicate and collaborate to guarantee that the software system under construction satisfies user needs and is in line with the company’s overall goals.

CONCLUSION

- You can use a range of techniques for requirements elicitation including interviews, scenarios, use-cases and ethnography.
- Requirements validation is the process of checking the requirements for validity, consistency, completeness, realism and verifiability.
- Business, organizational and technical changes inevitably lead to changes to the requirements for a software system. Requirements management is the process of managing and controlling these changes.


LUT
University